

CSCI1103 · PROGRAMMING PRINCIPLES I

Numeric Systems, Bits and Bitwise Operators

03

Beneath every int and char is a row of bits — binary and hexadecimal notation, and the operators that read and rewrite them directly.

LEARNING OBJECTIVES

Learning objectives

By the end of this lecture, you will be able to:

- 01 Convert numbers between decimal, binary and hexadecimal in both directions.
- 02 Apply the bitwise operators AND, OR, XOR and NOT at the bit level.
- 03 Use left/right shift to multiply, divide, and reposition bits.
- 04 Build and use a bitmask to set, clear, and check individual bits.
- 05 Count the number of set bits in an integer (Hamming weight).

NUMERIC SYSTEMS

Decimal, binary, hexadecimal

- ◆ Decimal (base 10): the digits 0–9 — what humans use.
- ◆ Binary (base 2): only `0` and `1` — what the hardware actually stores.
- ◆ Hexadecimal (base 16): digits 0–9 then A–F — a compact way to write binary (each hex digit = 4 bits).
- ◆ C++ literals: `0b1010` (binary), `0x1A` (hex), plain `26` (decimal).

CONVERTING

Any base → decimal

- ◆ Multiply each digit by its place value (a power of the base) and add them up.
- ◆ `1011` (binary) = $1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 = 11$
- ◆ `2F` (hex) = $2 \cdot 16 + 15 \cdot 1 = 47$

BINARY TO DECIMAL, BY HAND

```
1011 (base 2)
= 1*2^3 + 0*2^2 + 1*2^1 + 1*2^0
= 8 + 0 + 2 + 1 = 11
```

CONVERTING

Decimal → any base

- ◆ Repeatedly divide by the target base and collect the remainders.
- ◆ Read the remainders from last to first to get the final digits.

11 (DECIMAL) TO BINARY

```
11 / 2 = 5 remainder 1
 5 / 2 = 2 remainder 1
 2 / 2 = 1 remainder 0
 1 / 2 = 0 remainder 1
=> read bottom-up: 1011
```

WORTH THE REP

Do one conversion each direction by hand before you code it. If you can trace it on paper, you'll debug bit problems far faster than if you only memorized the shortcut.

BIT-LEVEL LOGIC

Bitwise operators

- ◆ `&` AND — 1 only where both bits are 1.
- ◆ `|` OR — 1 where at least one bit is 1.
- ◆ `^` XOR — 1 where the bits differ.
- ◆ `~` NOT — flips every bit.
- ◆ Not the same symbols as the logical operators from lecture 2 (`&&` `||` `!`) — those work on whole booleans, these work per bit.

```
int a = 0b1100;
int b = 0b1010;
a & b; // 0b1000
a | b; // 0b1110
a ^ b; // 0b0110
```

REPOSITIONING BITS

Left shift and right shift

- ◆ `x << n` shifts bits left by n places — equivalent to multiplying by 2^n .
- ◆ `x >> n` shifts bits right by n places — equivalent to dividing by 2^n (for non-negative x).
- ◆ Shifting is often faster than multiply/divide, and shows up constantly in flags and masks.

```
int x = 5;           // 0b0101
x << 1;              // 0b1010 = 10
x >> 1;              // 0b0010 = 2
```

WORKING WITH INDIVIDUAL BITS

Bit masking

- ◆ A mask is an integer whose bits mark which positions to touch, e.g. `1 << k` marks only bit k.
- ◆ Set bit k: `x = x | (1 << k);`
- ◆ Clear bit k: `x = x & ~(1 << k);`
- ◆ Check bit k: `(x >> k) & 1` is 1 if that bit is set, else 0.

SET, CLEAR, CHECK BIT 2

```
int x = 0b0000;
x |= (1 << 2);           // set    -> 0b0100
bool set = (x >> 2) & 1; // check -> true
x &= ~(1 << 2);          // clear -> 0b0000
```

BONUS APPLICATION

Counting set bits (Hamming weight)

- ◆ Counts how many bits equal 1 in an integer's binary form.
- ◆ Simple loop version: check the lowest bit, then right-shift, until the number reaches 0.

A BEGINNER-FRIENDLY HAMMING WEIGHT

```
int countSetBits(int n) {  
    int count = 0;  
    while (n > 0) {  
        count += (n & 1);  
        n >>= 1;  
    }  
    return count;  
}
```

RECAP

Key takeaways

- ✓ Any integer can be written in decimal, binary or hex — same value, different notation.
- ✓ Bitwise `& | ^ ~` and shifts `<< >>` operate on individual bits, not whole values.
- ✓ A mask built from `1 << k` sets, clears or checks bit k.
- ✓ These operators are the last new syntax before Quiz 1 — everything from here builds on lectures 1-3.

STUDIO TIME

Laboratory work #3

Convert between numeral systems by hand and in code, then practice masking and counting bits.

- ◆ Write conversion functions between decimal, binary and hexadecimal.
- ◆ Implement set/clear/check-bit helpers using masks.
- ◆ Implement and test countSetBits on several inputs.

Bitwise operators aren't on Quiz 1 (lectures 1-2), but they are on Quiz 2 – get this lab solid now so you're not relearning it under time pressure.

UP NEXT

Next: What is an array?

After Quiz 1 in week 4, lecture 4 introduces arrays — the first data structure that holds many values under one name.

L4 – What is an array? · Week 5 (after Quiz 1 in week 4)